

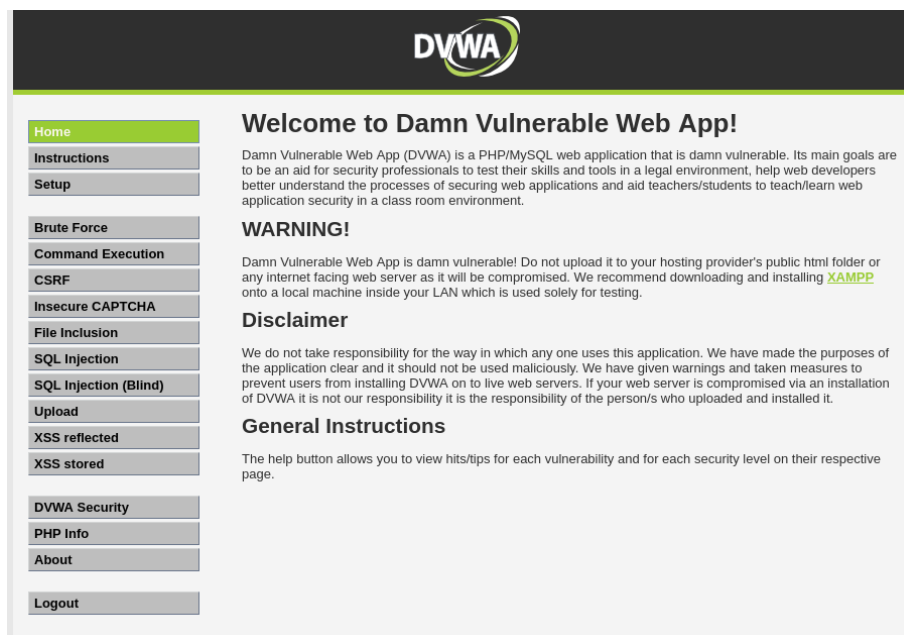
## LAB 5: Web Application Penetration Testing

|            |                                 |
|------------|---------------------------------|
| Name:      | Keertennah Devi A/P Ponnambalam |
| Matric No: | MEC254026                       |
| Section:   | 52                              |

Web applications is the most used piece of software in the world due to its of use and access. However, because of its highly accessible feature, it makes it a prime target for malicious users to break into the sensitive areas of the web application without authorization to steal data.

In this lab, you will practice in exploiting the security vulnerabilities on the DVWA web application.

1. Start the OWASP Broken Web Application (BWA) VM server. And login into the Damn Vulnerable Web Application (DVWA) module.



## LAB 5: Web Application Penetration Testing

2. The Damn Vulnerable Web Application (DVWA) contains lists of vulnerabilities that you can attempt to exploit. Attempt to exploit the following vulnerability and record your actions.

- a. Exploit the brute force password vulnerability via the fuzzing technique to gain access to the web application.

- i. How long did the exploitation take to get the password?

The exploitation took around 5 min to identify the correct password which is 'admin'.

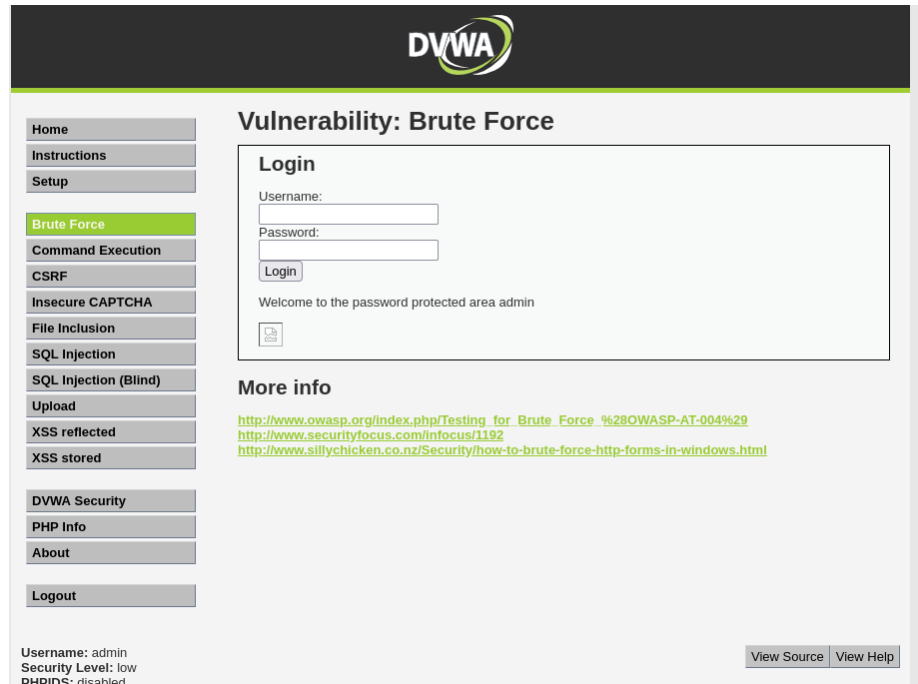


Figure 2: Choose Bruce force interface for fuzzing the password field.

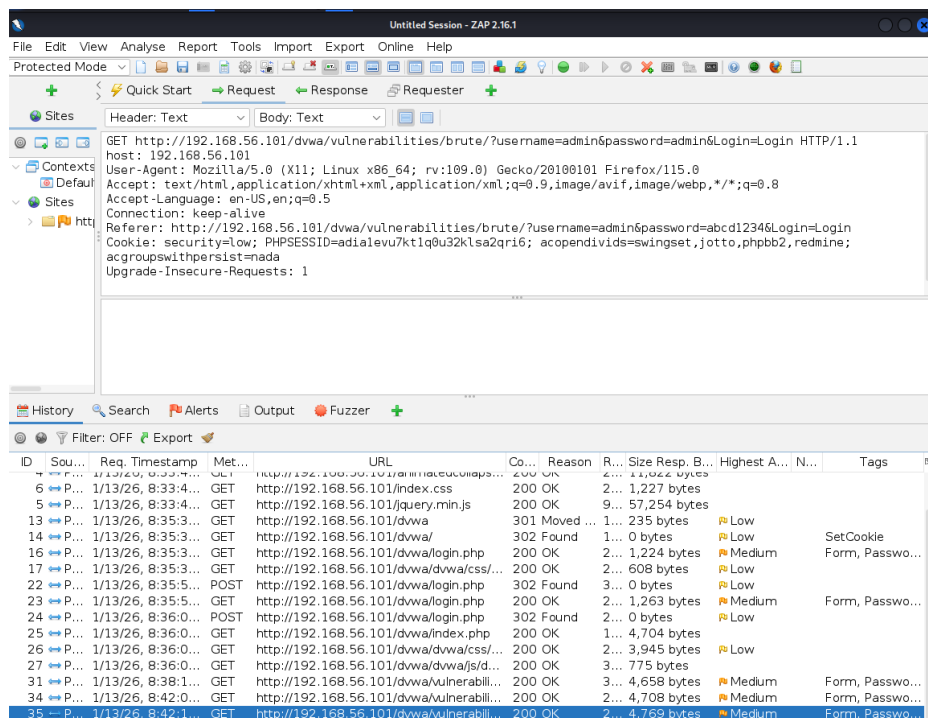


Figure 3: Fuzzing attack in progress with multiple requests.

## LAB 5: Web Application Penetration Testing

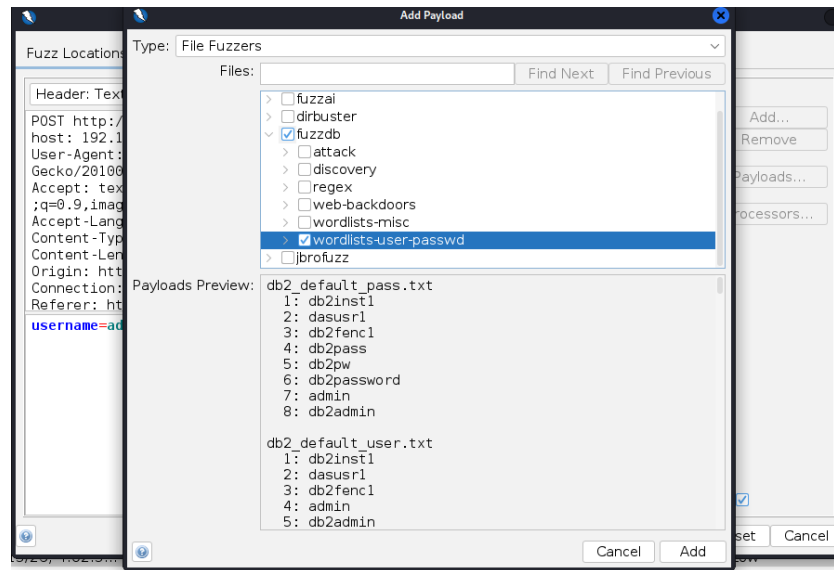


Figure 4: Add and choose wordlists-user-passwd payload to fuzz

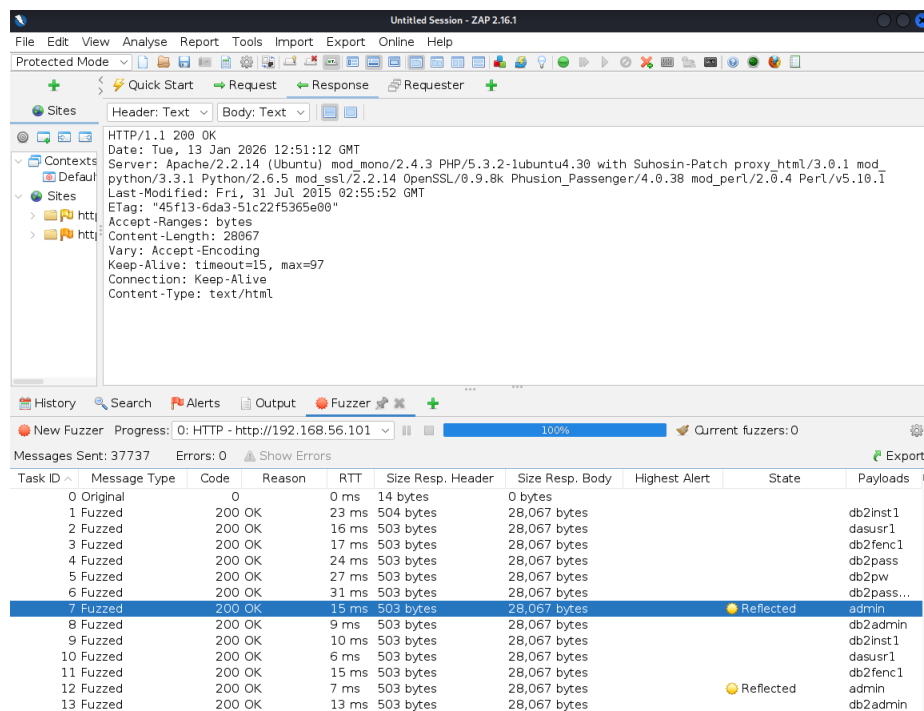


Figure 5: Results showing password (admin) fuzzed

### ii. How to mitigate the vulnerability?

We can mitigate the vulnerability by implementing temporarily account lookout after 3 to 5 failed login attempts or when not in use. Next, we can also increase delay time between login attempts to slow down attacks. In addition, we can ensure users set strong and complex password that include length, mixed cases, numbers and symbols and not allow browsers to remember passwords. The most mitigation method is to generate unique random tokens for each session request or ID. A request that doesn't match with session ID token is prevent from reach out an application. For more effectiveness, implementing CAPTCHA help to avoid any bot to login into system as humans. Another method is to track

## LAB 5: Web Application Penetration Testing

failed login attempts and alert users or admins of unusual activity. Finally, we can enforce to use multi-factor authentication (MFA) for safety purposes. These methods will help to reduce the risk of attacks on the system.

- b. Exploit the SQL Injection vulnerability via:
  - i. By inputting a malicious SQL Injection code manually and record the results.



Figure 6: Manual SQL injection input (1' OR '1'='1) in DVWA's SQL Injection page and the result displaying all user records from the database.

- ii. By fuzzing the input

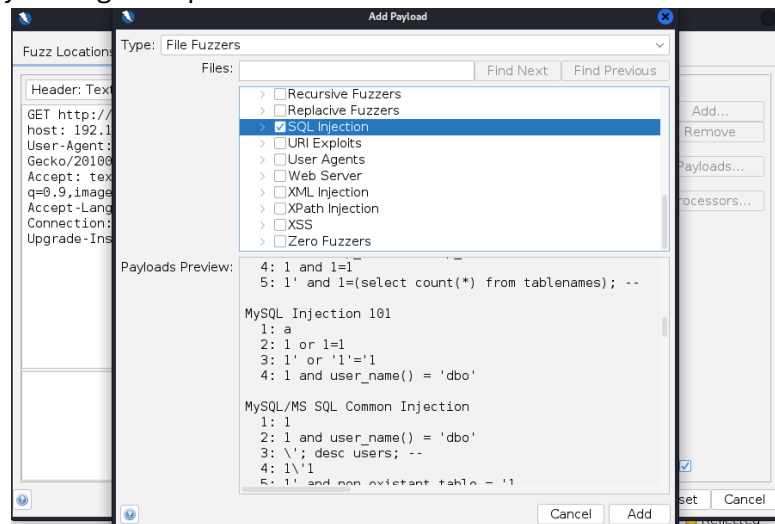


Figure 7: ZAP Fuzzer configuration for SQL injection testing

## LAB 5: Web Application Penetration Testing

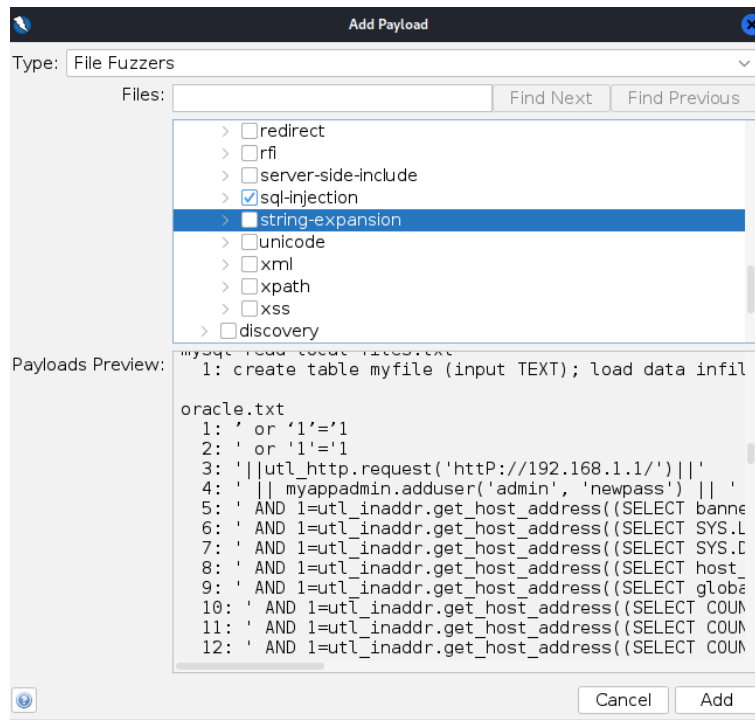
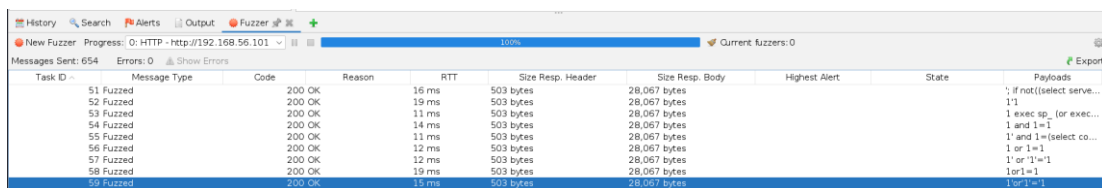


Figure 8: List of SQL payloads loaded from FuzzDB.



| Task ID   | Message Type | Code   | Reason | RTT       | Size Resp. Header | Size Resp. Body | Highest Alert | State | Payloads               |
|-----------|--------------|--------|--------|-----------|-------------------|-----------------|---------------|-------|------------------------|
| 51 Fuzzed | 200 OK       | 200 OK | 16 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | ' or '1'='1            |
| 52 Fuzzed | 200 OK       | 200 OK | 19 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1'                     |
| 53 Fuzzed | 200 OK       | 200 OK | 11 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1 exec sp_ (or exec... |
| 54 Fuzzed | 200 OK       | 200 OK | 14 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1 and 1=1              |
| 55 Fuzzed | 200 OK       | 200 OK | 11 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1' and 1=(select co... |
| 56 Fuzzed | 200 OK       | 200 OK | 12 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1 or 1=1               |
| 57 Fuzzed | 200 OK       | 200 OK | 12 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1' or '1'='1           |
| 58 Fuzzed | 200 OK       | 200 OK | 19 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1 or 1=1               |
| 59 Fuzzed | 200 OK       | 200 OK | 15 ms  | 503 bytes | 28,067 bytes      | 28,067 bytes    |               |       | 1' or '1'='1           |

Figure 9: Fuzzing results showing successful SQL payloads.

### iii. Screenshot and explain your results.



Figure 10: Detailed response of a successful SQL injection request.

## LAB 5: Web Application Penetration Testing

SQL Injection is easily exploited when user input is not filtered or sanitized. As shown in the screenshots, an attacker can abuse the input field by using the ' symbol to break out of the intended data string. By adding true where clause, they can manipulate the query to force the database to return all stored records to the user.

- c. Exploit the XSS Reflected vulnerability via:
  - i. Inputting malicious JavaScript code manually and record the results.

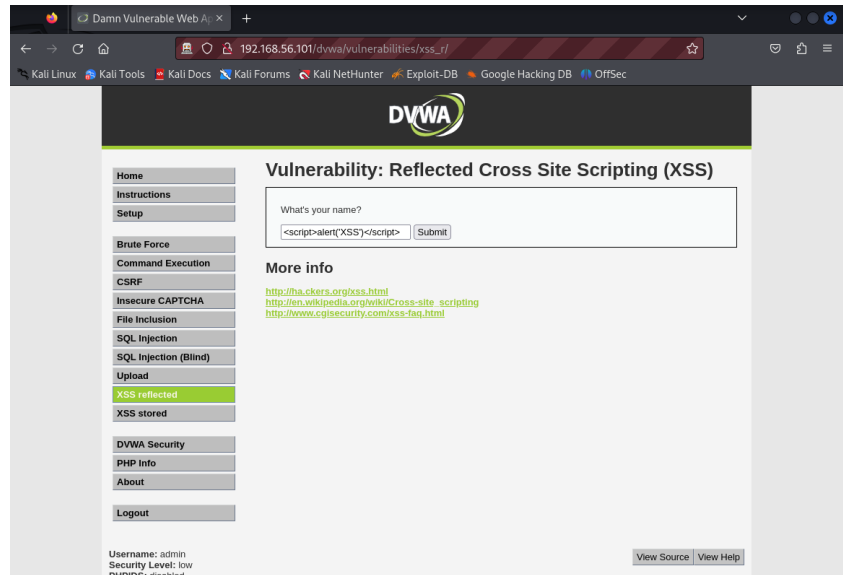


Figure 11: Input field with XSS payload: `<script>alert('XSS')</script>`

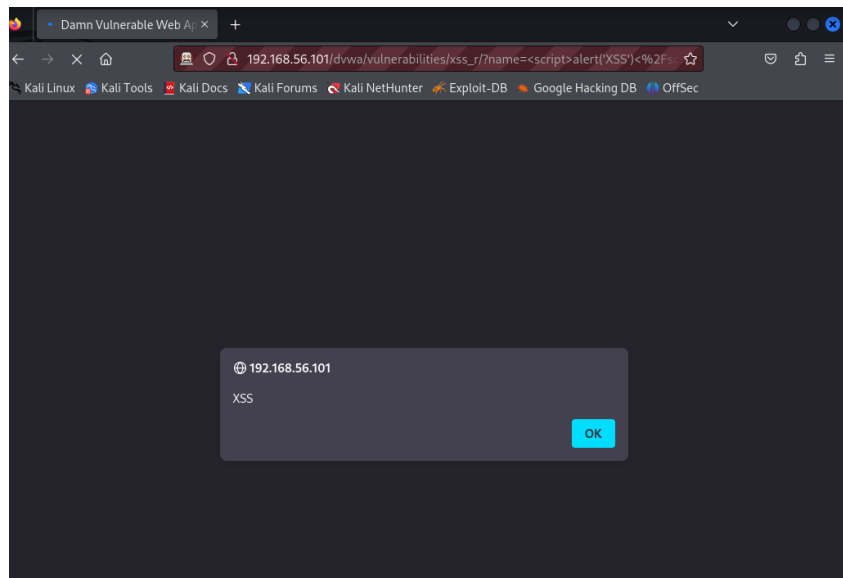


Figure 12: Alert pop-up showing successful XSS execution.

## LAB 5: Web Application Penetration Testing

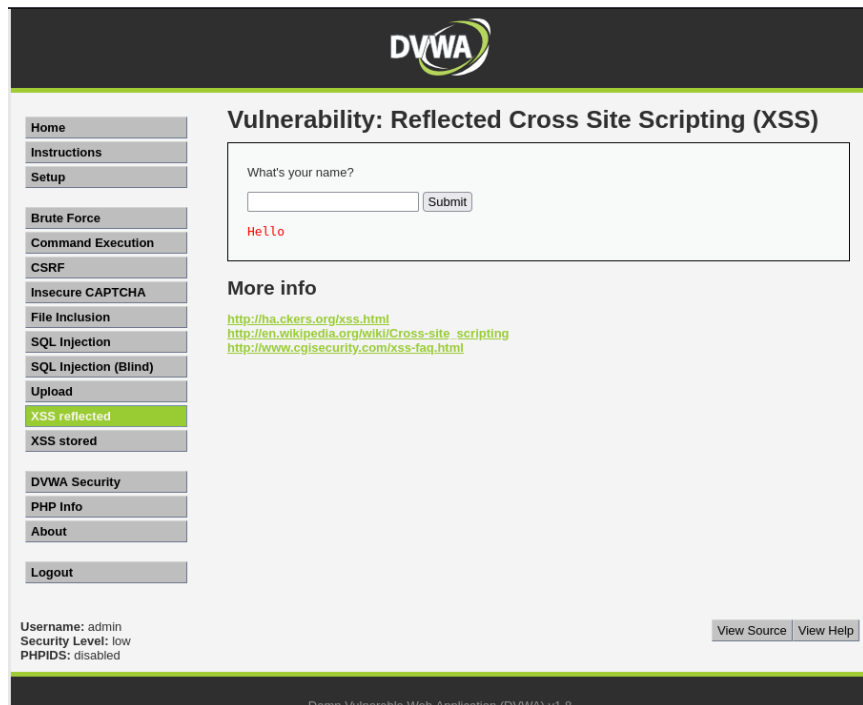


Figure 13: “Hello” appears in the box after clicking OK

ii. Fuzzing the input.

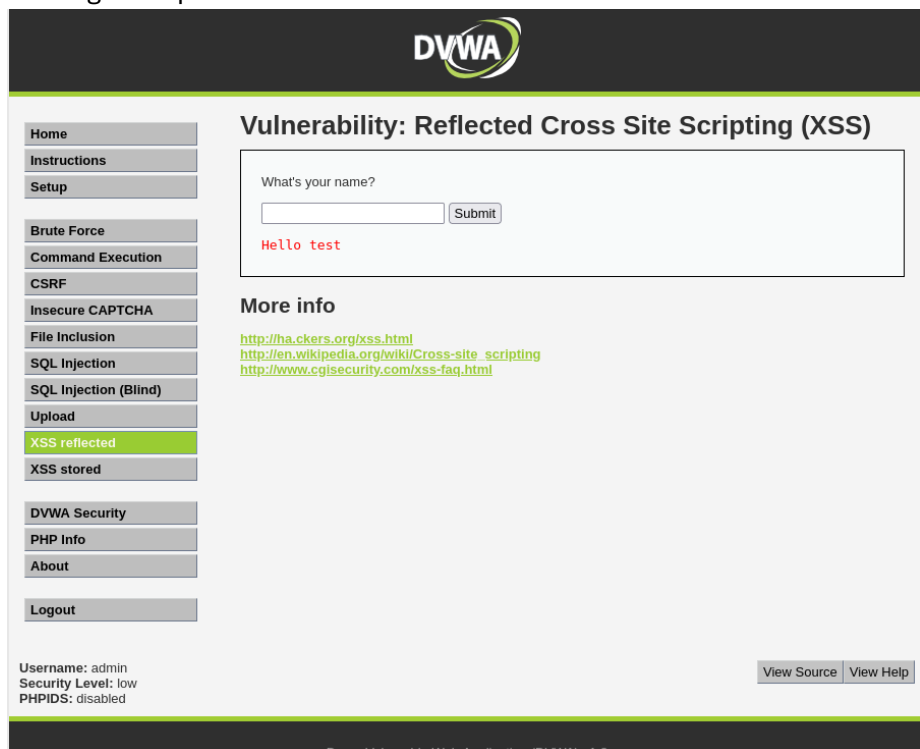


Figure 13: Add the word “test” for XSS testing.

## LAB 5: Web Application Penetration Testing

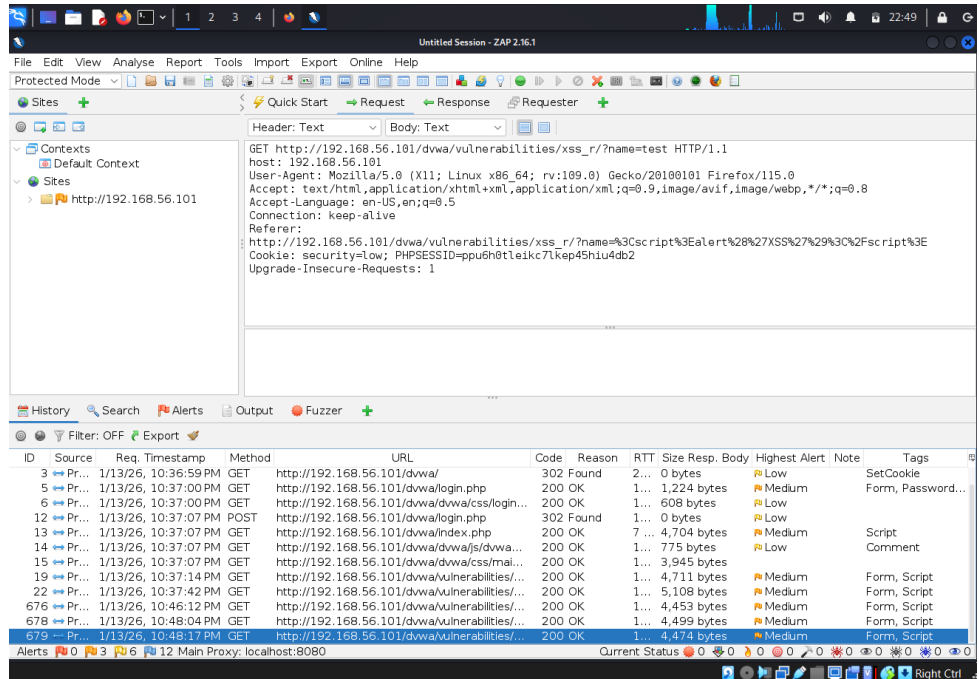


Figure 14: Fuzzing results with various XSS payloads.

| Task ID | Message Type | Code   | Reason | RTT   | Size Resp. Header | Size Resp. Body | Highest Alert | State | Payloads    |
|---------|--------------|--------|--------|-------|-------------------|-----------------|---------------|-------|-------------|
| 0       | Original     | 0      |        | 0 ms  | 14 bytes          | 0 bytes         |               |       |             |
| 1       | Fuzzed       | 200 OK |        | 13 ms | 503 bytes         | 28,067 bytes    |               |       | <script>... |
| 2       | Fuzzed       | 200 OK |        | 14 ms | 504 bytes         | 28,067 bytes    |               |       | <img sc...  |
| 3       | Fuzzed       | 200 OK |        | 14 ms | 503 bytes         | 28,067 bytes    |               |       | "><scrip... |
| 4       | Fuzzed       | 200 OK |        | 13 ms | 503 bytes         | 28,067 bytes    |               |       | <svg onl... |

Figure 15: Successful XSS payload shows fuzzed

- iii. Attempt post-exploitation e.g., Obtain the session ID.

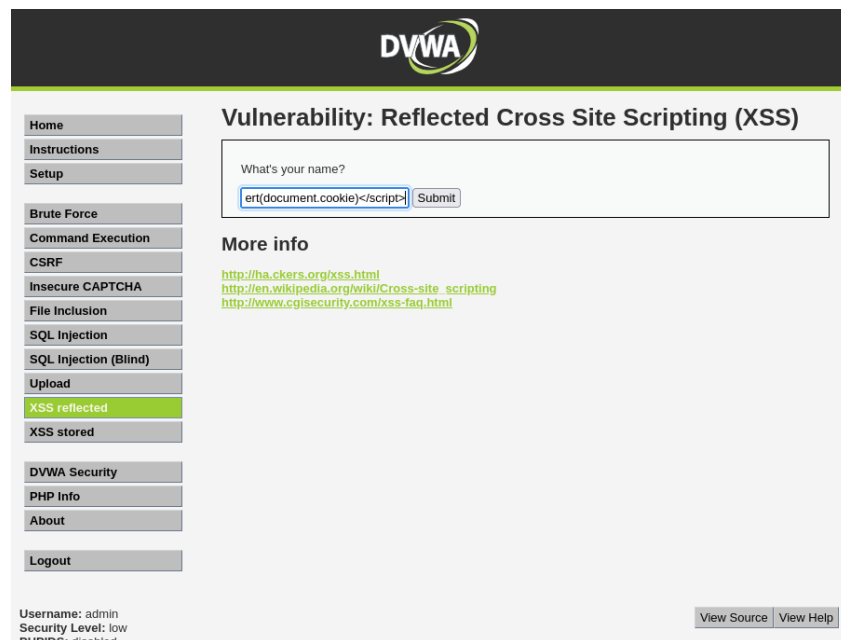


Figure 16: Add `<script>alert(document.cookie)</script>` into the empty field



## LAB 5: Web Application Penetration Testing

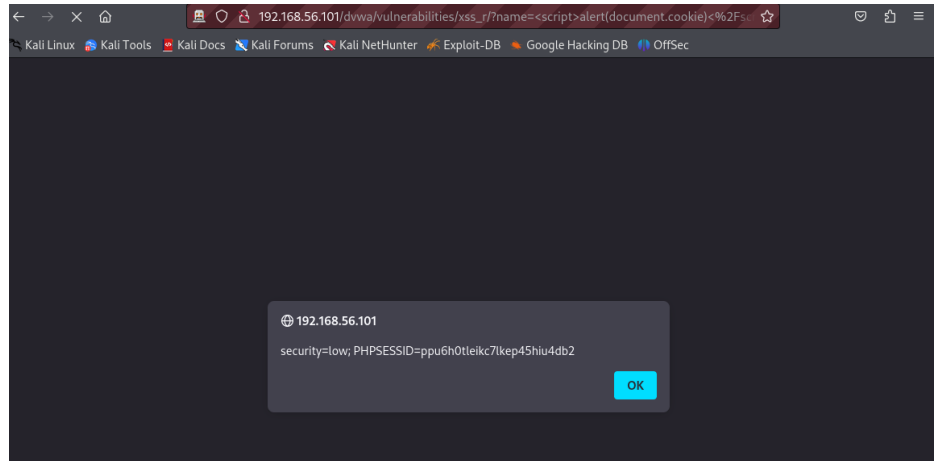


Figure 17: The pop-up box shows the PHPSESSID

- d. Exploit the XSS – Stored vulnerability via:
  - i. Inputting malicious JavaScript code (manually) and record the results.

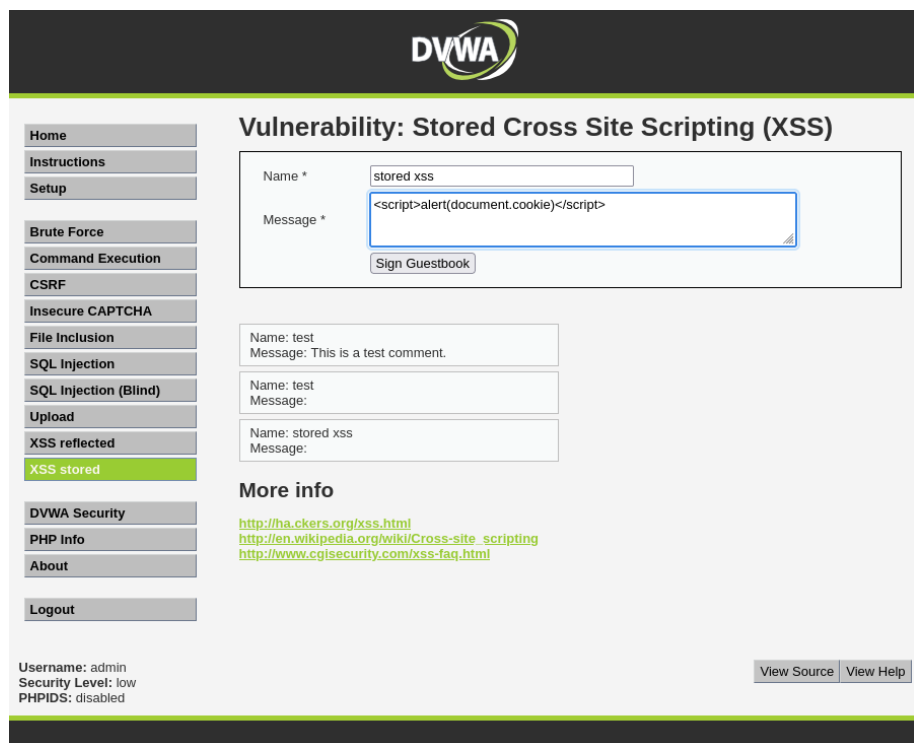


Figure 18: Stored XSS script into a message field in DVWA



Figure 19: The stored script displayed on the page.

## LAB 5: Web Application Penetration Testing

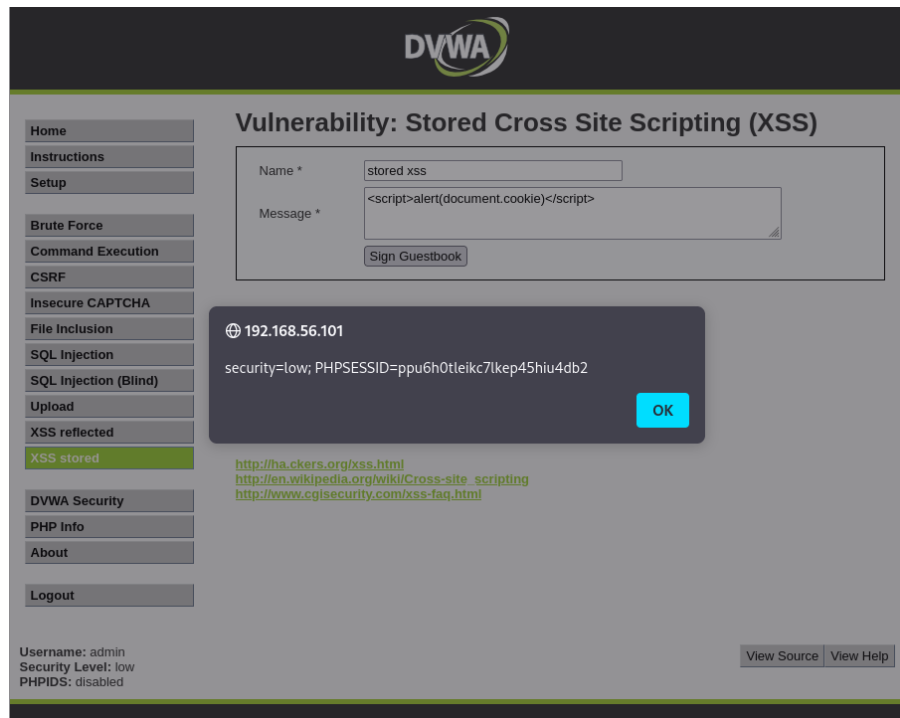


Figure 20: Alert pop-up when visiting the page with the stored XSS.

### ii. Fuzzing the input.

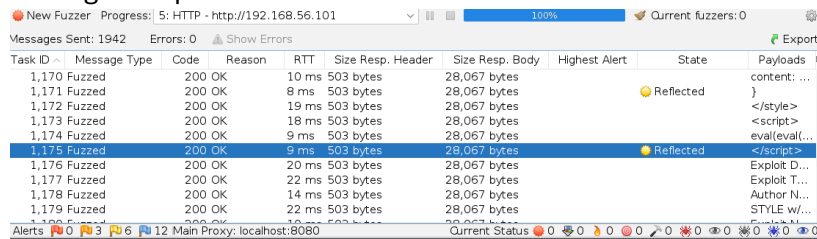


Figure 21: Fuzzing results for stored XSS vulnerabilities.

### iii. Attempt post-exploitation using the Beef-XSS tool.

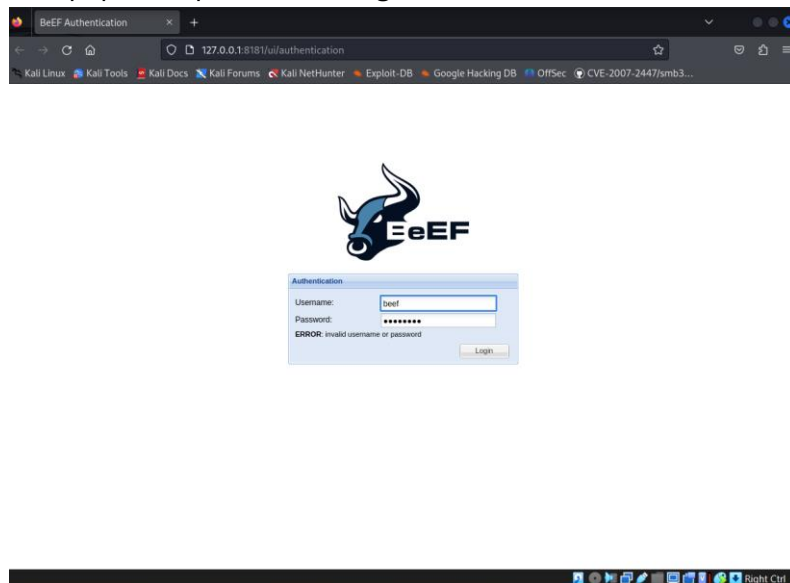


Figure 22: Beef-XSS control panel login.

## LAB 5: Web Application Penetration Testing

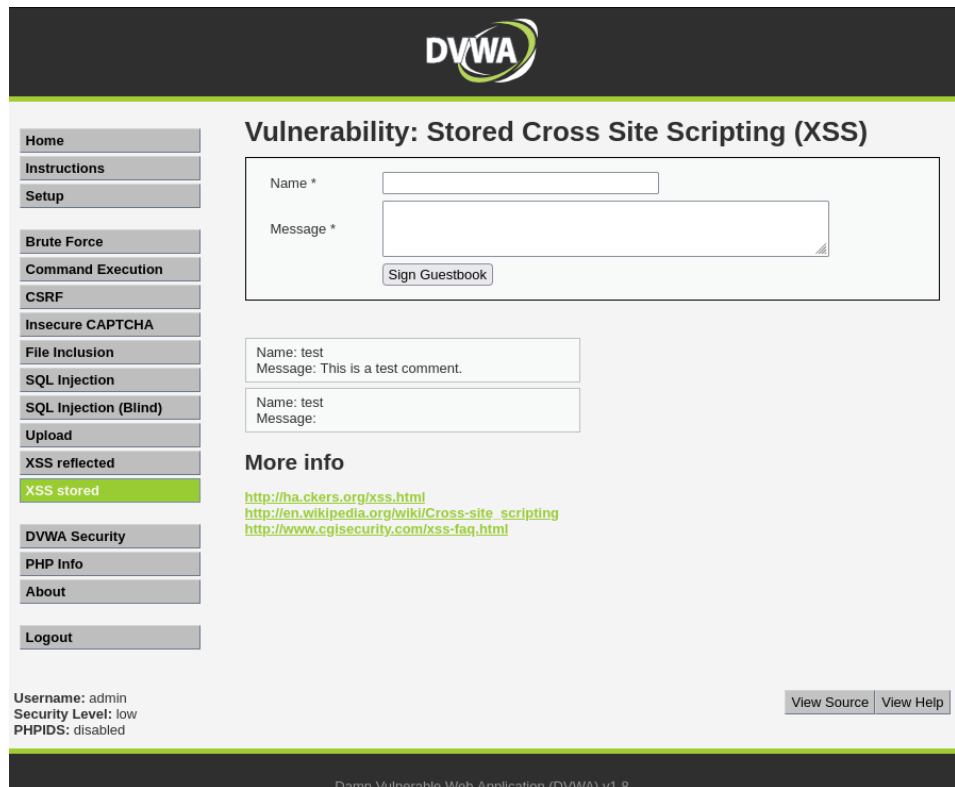


Figure 23: Injected Beef script in DVWA stored XSS field.

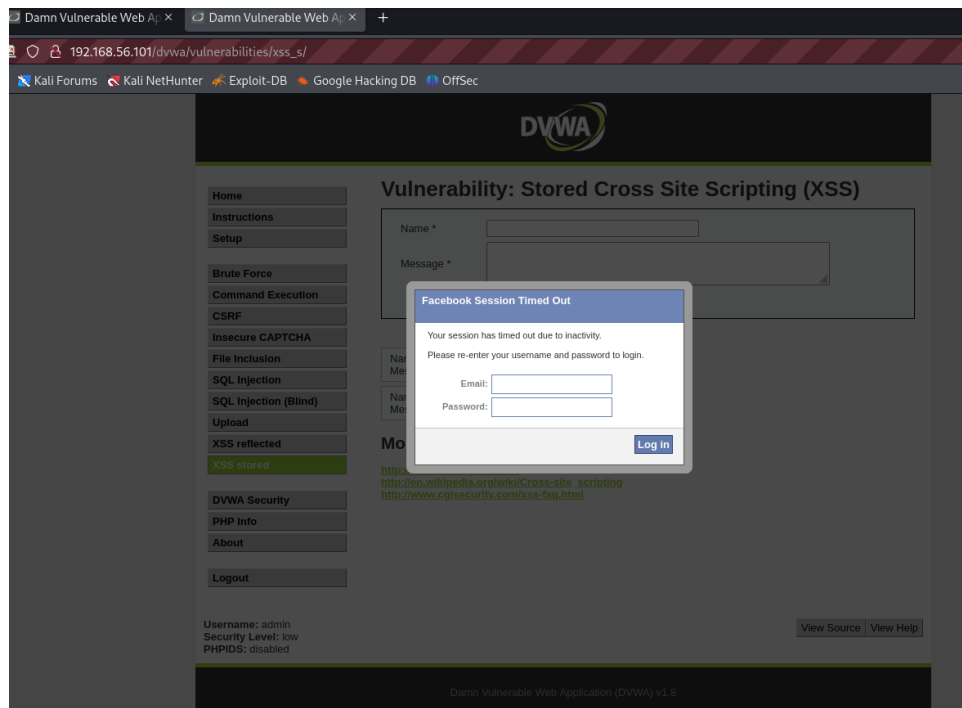


Figure 24: Executed Facebook login from Beef and Pop-up box appear in DVWA.

## LAB 5: Web Application Penetration Testing

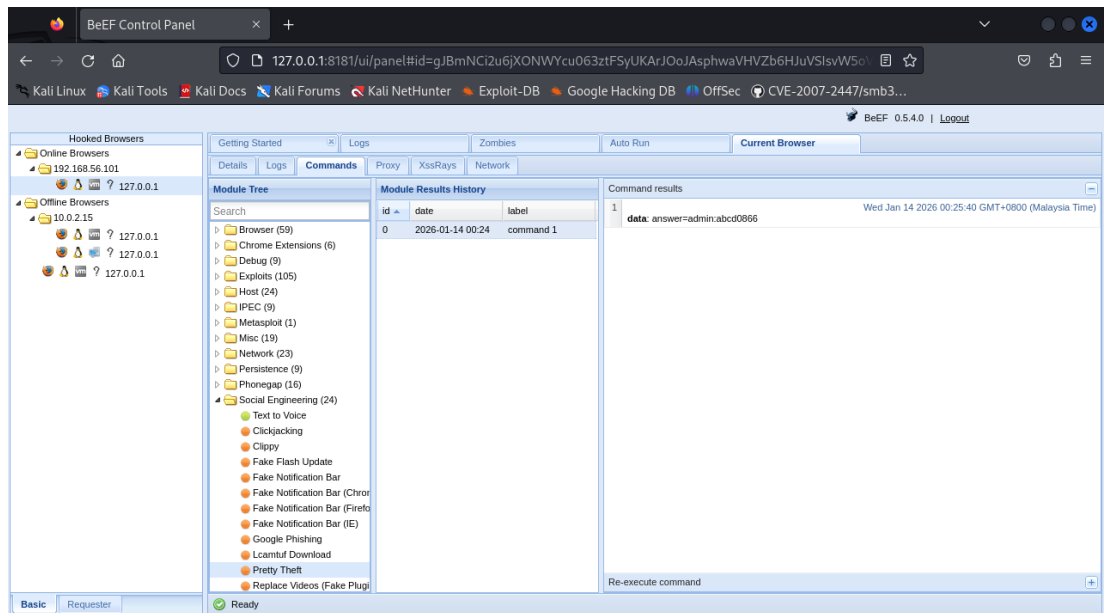


Figure 25: Beef command module (Social Engineering → Pretty Theft) and captured credentials (Facebook login) shown in Beef.

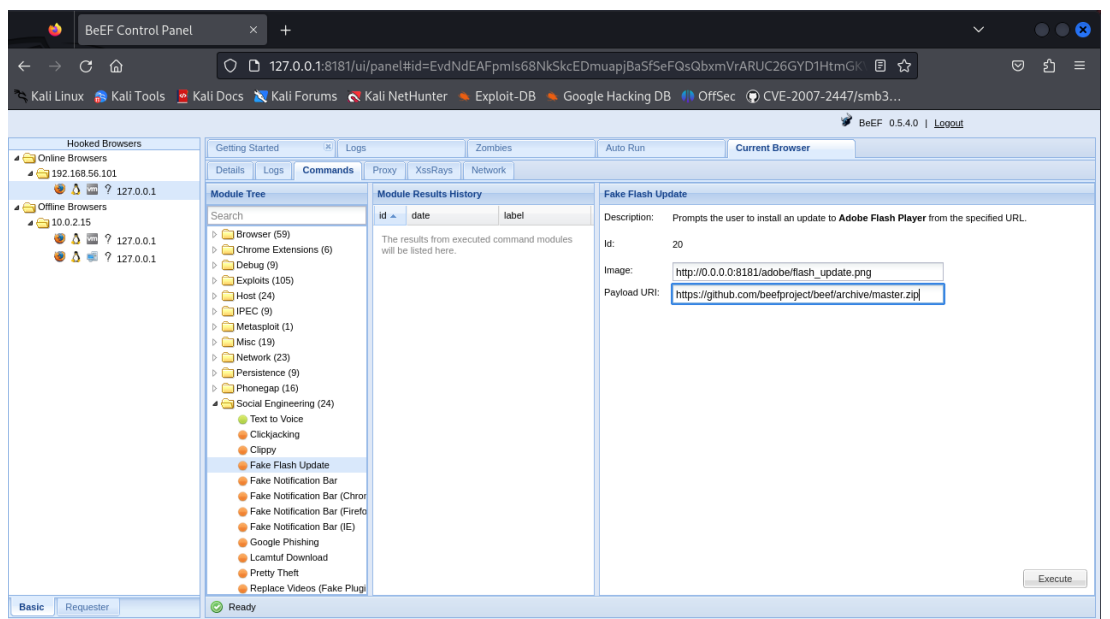


Figure 26: Beef command module (Social Engineering → Fake Flash Update)

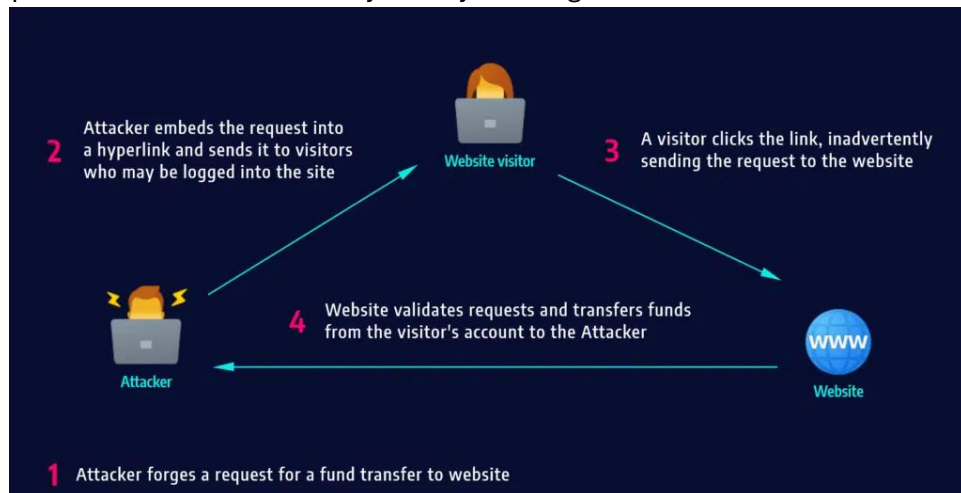
## LAB 5: Web Application Penetration Testing



Figure 27: Fake login prompt displayed on browser.

### 3. [Bonus] Attempt to exploit the CSRF vulnerability to change the password.

- a. Explain what CSRF vulnerability is. May use a figure.

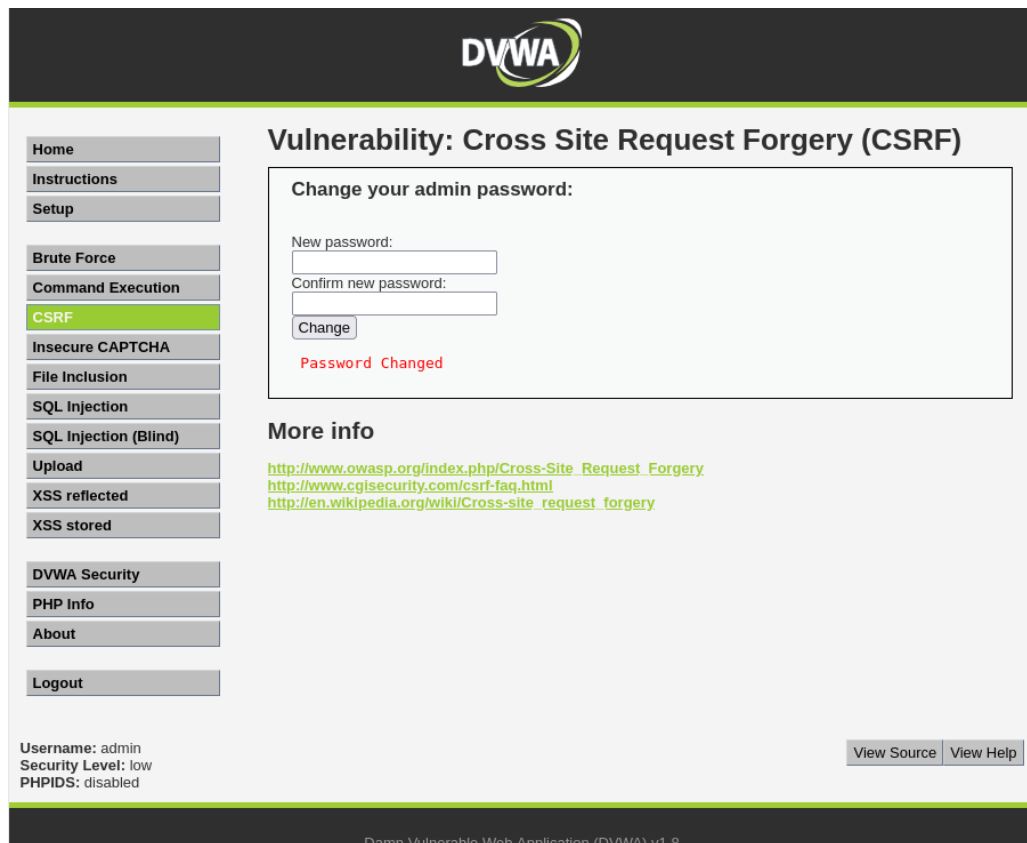


Cross-Site Request Forgery (CSRF) is an attack where a malicious website tricks a logged-user into unknowingly submitting a request to a vulnerable web application. The attacker detects a specific action on the target website such as a bank transfer or password change. They "forge" the request for this action. For example, an encoded URL. The attacker sends a fake request to a Website Visitor (victim). This is typically hidden as a safe link in an email, chat message, or even embedded in a malicious graphic on a different website. The victim chooses to click the link. Because the victim has previously logged into the "Website", their browser automatically adds their session cookies in the request. The request is

## LAB 5: Web Application Penetration Testing

delivered to the intended website. Because it finds valid session cookies, it concludes that the victim took the action on purpose. The website validates the request and, in this example, executes the command, transferring funds to the attacker. The "vulnerability" exists because the website just uses automatically submitted credentials (such as cookies) to verify requests. It cannot distinguish between a request that the user meant to make and one that was initiated by a malicious third party. CSRFs are often carried out using malicious social engineering, such as an email or link that tricked the victim into submitting a malicious request to a server. As the unknowing user is authenticated by their application at the time of the attack, it is impossible to know the difference between a valid and fake request.

- b. Exploit the CSRF vulnerability. There are a few ways that you can attempt to do it.  
[Hint: Craft your own request URL to change the password]



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The top navigation bar includes links for Home, Instructions, Setup, Brute Force, Command Execution, CSRF (highlighted in green), Insecure CAPTCHA, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected, XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: Cross Site Request Forgery (CSRF)" and contains a form to "Change your admin password:". The form has two input fields: "New password:" and "Confirm new password:", followed by a "Change" button. Below the button, a red message states "Password Changed". Under the "More info" section, there are three links: [http://www.owasp.org/index.php/Cross-Site\\_Request\\_Forgery](http://www.owasp.org/index.php/Cross-Site_Request_Forgery), <http://www.cgisecurity.com/csrf-faq.html>, and [http://en.wikipedia.org/wiki/Cross-site\\_request\\_forgery](http://en.wikipedia.org/wiki/Cross-site_request_forgery). At the bottom left, the user status is displayed: "Username: admin", "Security Level: low", and "PHPIDS: disabled". At the bottom right, there are "View Source" and "View Help" buttons. The footer text reads "Damn Vulnerable Web Application (DVWA) v1.8".

Figure 28: DVWA CSRF vulnerability page with password change form from admin to abc123

## LAB 5: Web Application Penetration Testing

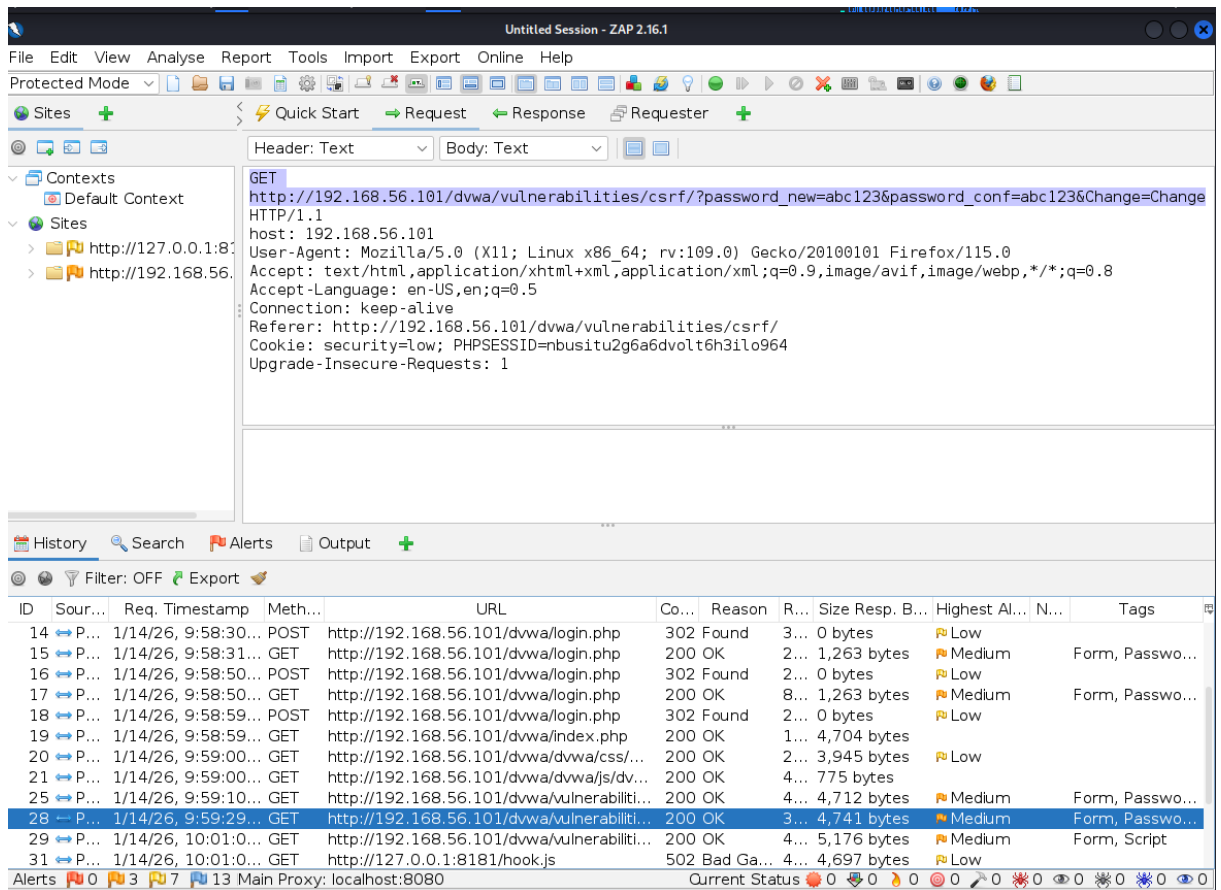


Figure 29: Found the old password that changed to abc123 in ZAP

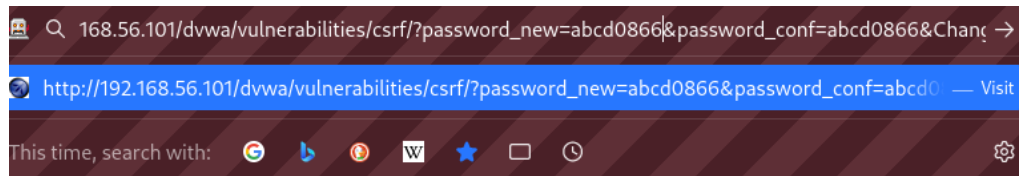


Figure 30: Crafted malicious URL for password change from abc123 (password\_new=abcd0866).  
Then, execute the CSRF attack

## LAB 5: Web Application Penetration Testing

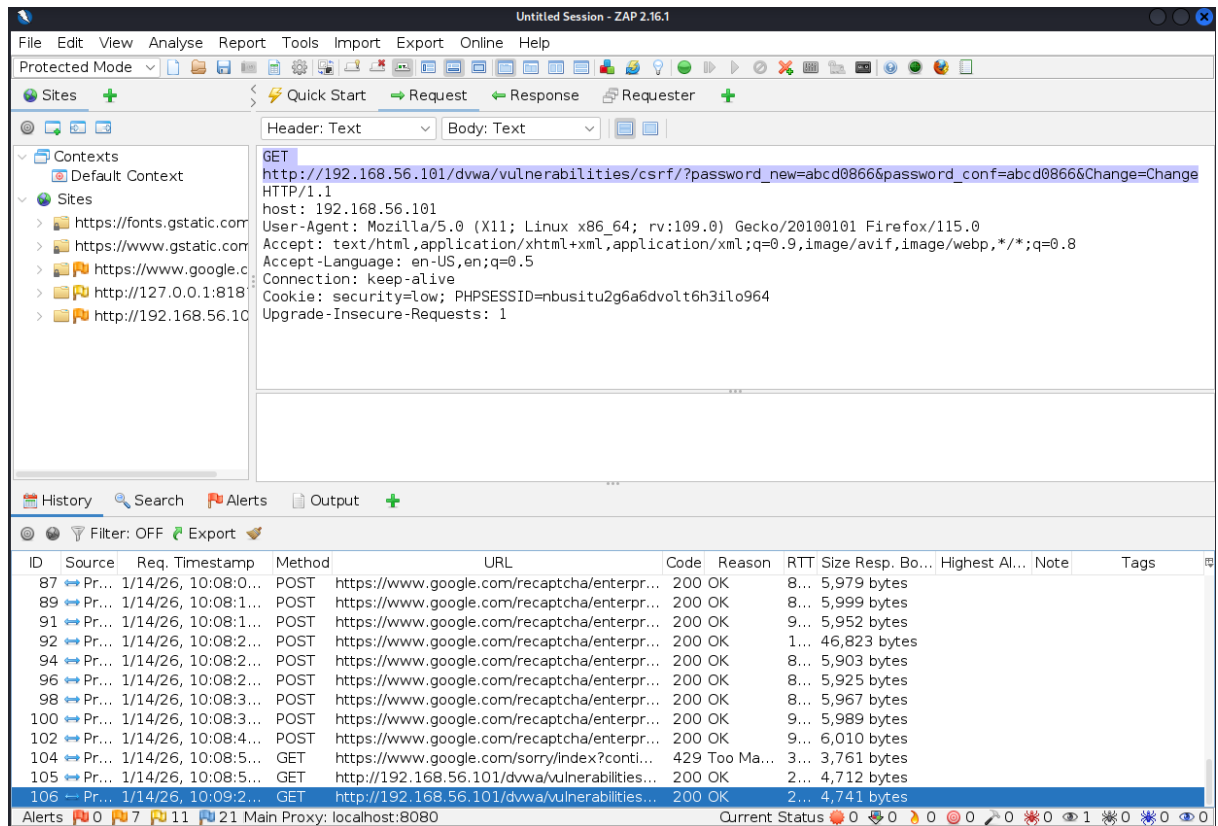


Figure 31: checked crafted URL password\_new to abcd0866

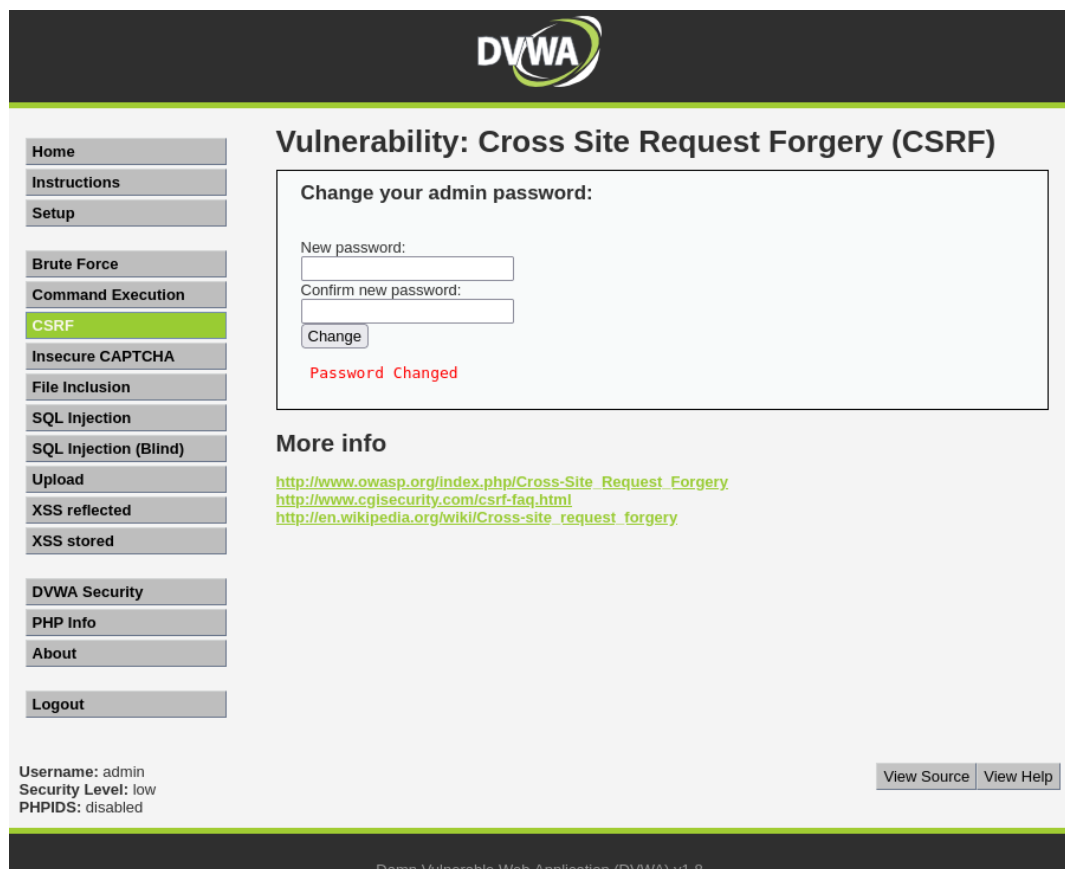


Figure 32: CSRF indicate success message confirming password changed to abcd0866



## LAB 5: Web Application Penetration Testing



Username

Password

You have logged out

Damn Vulnerable Web Application (DVWA) is a RandomStorm OpenSource project

Figure 33: Login DVWA with new password (abcd0866) to ensure that CSRF exploit is successfully



### Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goals are to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and aid teachers/students to teach/learn web application security in a class room environment.

#### WARNING!

Damn Vulnerable Web App is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing web server as it will be compromised. We recommend downloading and installing [XAMPP](#) onto a local machine inside your LAN which is used solely for testing.

#### Disclaimer

We do not take responsibility for the way in which any one uses this application. We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

#### General Instructions

The help button allows you to view hits/tips for each vulnerability and for each security level on their respective page.

You have logged in as 'admin'

Username: admin  
Security Level: low  
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.8

Figure 34: Successfully login into DVWA with new password